

Product Requirements Document

Multimodal Collaborative Code Review Sandbox

Version: 1.0

Status: Draft / Proposed

Date: July 2026

1. Executive Summary

The **Multimodal Collaborative Code Review Sandbox** is a web-based, real-time workspace designed to eliminate the friction in complex code reviews, pair-programming interviews, and technical alignment

huddles. By combining multi-user code editing, a secure execution terminal, and a visual drawing canvas layered directly over the code, the platform bridges the gap between abstract architectural discussions and concrete code implementation.

2. Problem Statement & Solution

The Problem

Developers currently rely on fragmented toolchains for collaborative problem solving. Explaining a complex data flow often requires screen-sharing (which is non-interactive for viewers), pasting code blocks in chat applications (which lacks execution context), or using heavy IDE plugins (which require extensive setup and lock teams into specific editor ecosystems).

The Solution

A zero-install web platform accessible via a single URL. Users drop into a synchronized workspace where they can write code together in real-time, instantly test execution, and visually sketch out system architectures, data flows, or memory maps directly on top of the code editor.

3. Target Personas

- **The Interviewer / Candidate:** Needs a stable, real-time environment to evaluate coding skills without local environment biases. Values the execution terminal to verify logic immediately.
- **The Staff / Principal Engineer:** Needs to explain complex architectural changes or unblock a junior developer. Values the visual canvas to draw dependency graphs or state machines over the implementation.
- **The Open-Source Maintainer:** Needs a disposable, shareable sandbox to reproduce bugs synchronously with contributors.

4. Functional Requirements

4.1 Synchronous Code Editor

- **Real-time Sync:** Sub-50ms latency for character keystrokes, managed via Conflict-free Replicated Data Types (CRDTs).
- **Multi-Cursor Awareness:** Display remote user cursors with distinct colors and name tags. Provide active text selection highlights for all connected users.
- **Editor Engine:** Leverage Monaco Editor for syntax highlighting, code folding, and basic IntelliSense for JavaScript/TypeScript and Python.
- **Multi-file Support:** Users can create, rename, delete, and switch between multiple files within a session namespace.

4.2 The Overlay Canvas (Architectural Mapping)

- **Layered Integration:** An interactive, transparent drawing canvas layered directly over the code editor.
- **Modal Interaction:** A UI toggle to switch between "Typing Mode" (events route to Monaco) and "Drawing Mode" (events route to the canvas).
- **Multi-user Whiteboarding:** Synchronized drawing strokes, shapes, and text annotations across all clients.
- **Anchor Points (Phase 2):** Ability to visually anchor a drawn arrow or shape to a specific line of code, ensuring the drawing moves if the line shifts.

4.3 Terminal Simulator (Mock Execution)

- **Integrated Web Terminal:** An Xterm.js instance docked at the bottom of the workspace.
- **Real-time Output Sync:** The output stream (stdout/stderr) from the backend is synchronized so all users see the exact same console logs simultaneously.
- **Shared Execution Control:** Any participant can trigger code execution (e.g., via a "Run" button) or interact with the terminal prompt.

5. Non-Functional Requirements (NFRs)

5.1 Security & Sandboxing

This is the highest priority NFR. Allowing arbitrary code execution presents a critical remote code execution (RCE) risk.

- All code execution must occur within strictly isolated, ephemeral Docker containers or microVMs (e.g., Firecracker).
- Containers must have zero network access to internal VPCs or metadata APIs.
- Aggressive resource limits: Max execution time (e.g., 5 seconds), CPU constraints, and RAM limits (e.g., 128MB per session).

5.2 Scalability & Architecture

Component	Technology	Scalability Approach
Frontend	Next.js, Monaco, React	Deployed to Vercel/CDN for edge caching of static assets.
Sync Engine	Yjs + Node.js WebSockets	Horizontally scalable WebSocket servers using Redis Pub/Sub backplane.
Data Persistence	PostgreSQL + S3	Session states flushed from RAM to Postgres upon session close.

6. Out of Scope (Phase 1)

- Full IDE features (e.g., global refactoring, advanced debugging breakpoints).
- Direct integration with GitHub/GitLab (cloning repositories or opening PRs directly).
- Video or voice calling functionality (users will rely on external tools like Zoom or Slack).